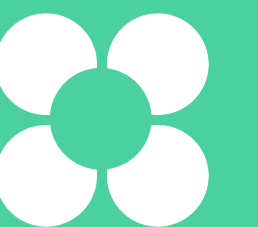


Система сбора логов Elastic Stack

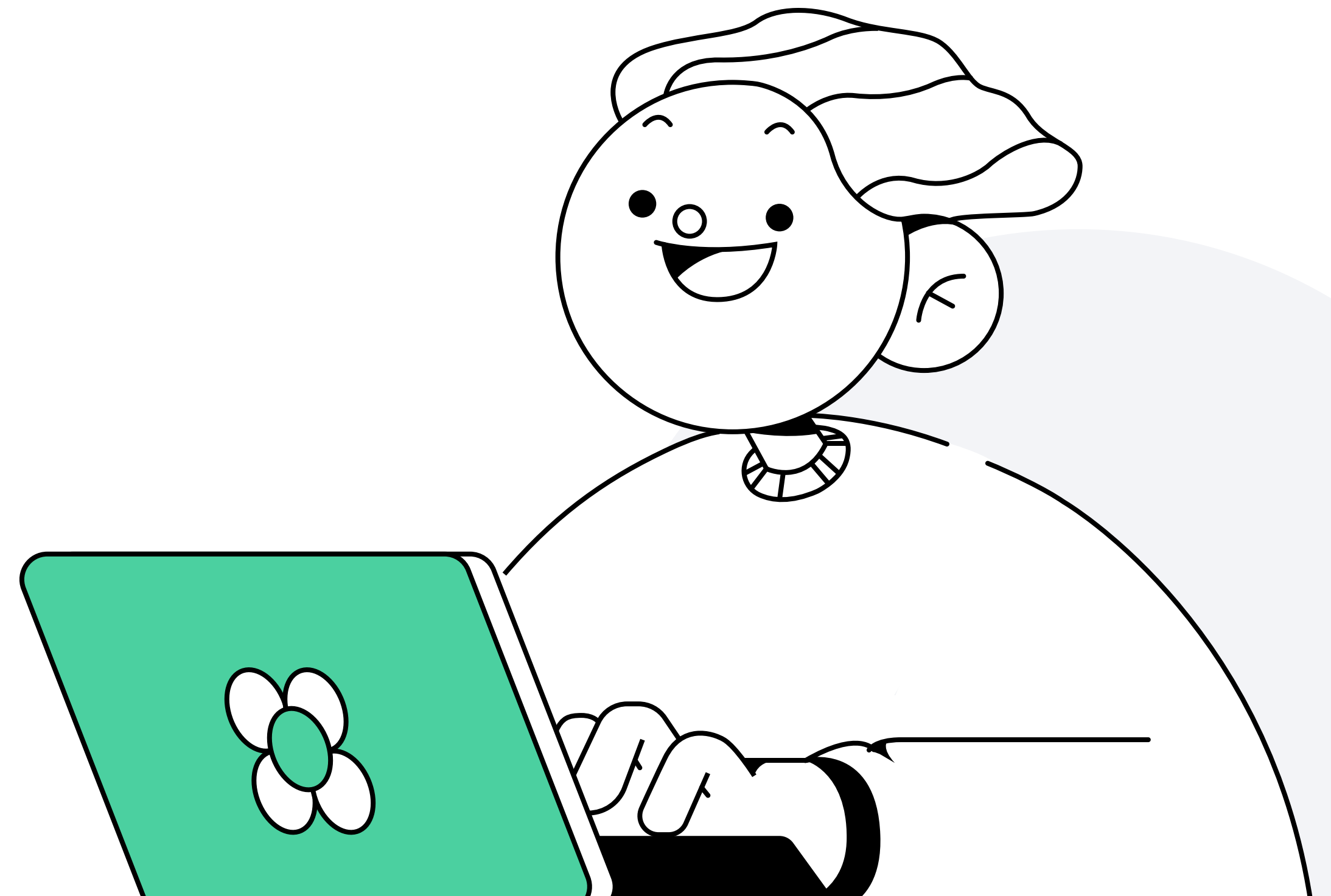
Сергей Бывшев

Ведущий инженер автоматизации в «Метре квадратном»



План занятия

- 1 Системы централизованного сбора логов
- 2 Filebeat
- 3 Logstash
- 4 Elasticsearch
- 5 Kibana



Системы централизованного сбора логов

Сергей Бывшев

Ведущий инженер автоматизации в «Метре квадратном»

Централизованное логирование

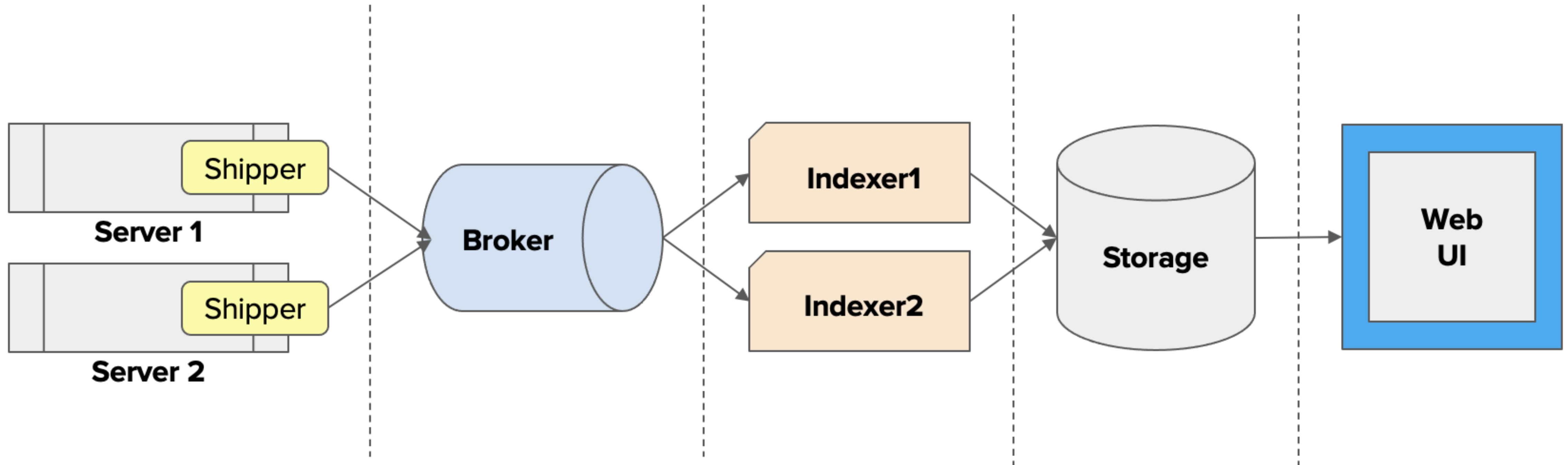
Централизованный сбор логов — один из важнейших разделов мониторинга в современном мире.

Состоит из нескольких этапов:

- отправка логов сервиса
- агрегация и индексация
- хранение
- визуализация

Централизованное логирование

Схема работы системы централизованного логирования:



Компоненты систем централизованного логирования

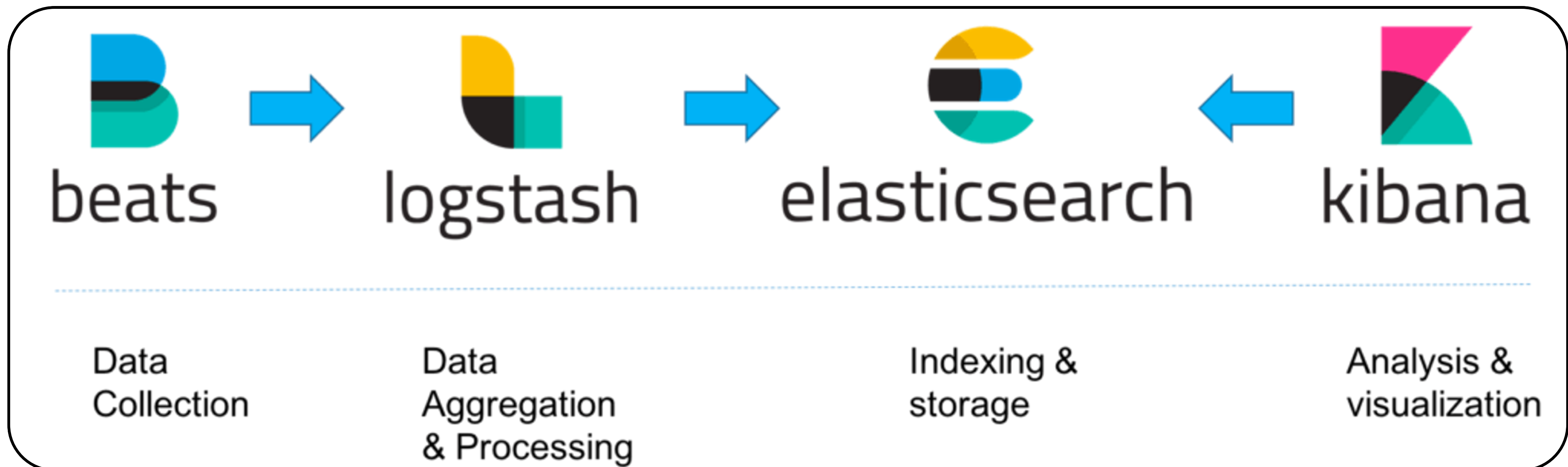
Основные компоненты систем централизованного логирования.

- **Shipper** — агент, работающий на одном хосте с сервисами, логи которых мы хотим собирать
- **Broker** — опциональный компонент, представляет собой очередь сообщений
- **Indexer** — занимается агрегацией, трансформацией и индексацией сообщений
- **Storage** — база данных, в которую сохраняются логи
- **Web UI** — веб-интерфейс для визуализации сообщений

ELK

Сейчас основное решение – система сбора логов, построенная на основе стека технологий **Elasticsearch**, **Logstash** и **Kibana (ELK)**:

Этот стек позволяет «из коробки» получить средство агрегации и трансформации, хранения и визуализации данных логирования



Filebeat

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Filebeat

Filebeat — эффективное средство сбора данных логирования из файлов и отправки в `logstash/elasticsearch`

В процессе сбора Filebeat может «схлопывать» данные логов при их последовательном повторе, хранить данные в буфере до восстановления конечной точки отправки, игнорировать определённые события логирования



Filebeat

Базовая конфигурация Filebeat довольно проста и заключается в настройке источника данных и конечной точки в конфигурационном файле **filebeat.yml**

Filebeat

Пример настройки чтения логов **nginx**:

```
- type: log
  enabled: true
  paths:
    - /var/log/nginx/access.log
  fields:
    service: nginx_access
  fields_under_root: true
  scan_frequency: 5s
```

Filebeat

Пример настройки отправки сообщений в **Logstash**:

```
output.logstash:  
  hosts: ["logstash_host:5044"]
```


Logstash

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Logstash

Logstash в Elastic-стеке сбора логов — инструмент для приёма данных, их преобразования и отправки в кластер БД Elasticsearch

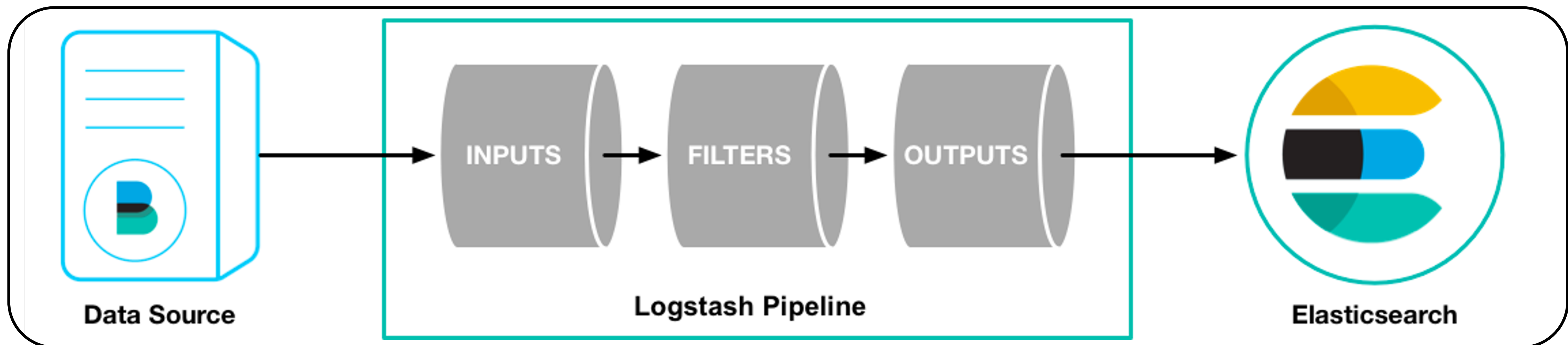
Конфигурирование Logstash осуществляется через специальный конфигурационный файл



Logstash

Обработка сообщений осуществляется с помощью пайплайнов, которые состоят из 3 секций:

- input
- filter
- output



Logstash

Графа `input` конфигуционного файла задаёт входные точки Logstash

Это может быть интерактивный обмен, сетевое соединение, файл и т. д.

Также указывается тип входных событий для их оптимальной обработки, например `syslog`

```
input {  
  tcp {  
    port => 5000  
    type => syslog  
  }  
  udp {  
    port => 5000  
    type => syslog  
  }  
}
```

Logstash

Графа filter отвечает за парсинг входных данных и преобразование их в необходимую структуру

Logstash имеет множество стандартных парсеров входных данных, но самый универсальный — grok. Но он наименее производительный.

В этой графе можно указать, в какие ключи складывать данные, вставлять унарные операторы в случае особых входных событий и т. д.

```
filter {  
  if [path] =~ "access" {  
    mutate { replace => { type => "apache_access" } }  
    grok {  
      match => { "message" => "%{COMBINEDAPACHELOG}" }  
    }  
    date {  
      match => [ "timestamp" , "dd/MMM/yyyy:HH:mm:ss Z" ]  
    }  
  } else if [path] =~ "error" {  
    mutate { replace => { type => "apache_error" } }  
  } else {  
    mutate { replace => { type => "random_logs" } }  
  }  
}
```

Logstash

Графа output отвечает за конечную точку для отправки данных из Logstash.

Можно указать несколько конечных точек для параллельной отправки данных: файлы, Elasticsearch.

Также для каждой конечной точки существуют дополнительные гибкие настройки: добавление даты в название файла или применение шаблона индексов в Elasticsearch к данным

```
output {  
  elasticsearch { hosts => ["localhost:9200"] }  
  stdout { codec => rubydebug }  
}
```


Elasticsearch

Сергей Бывшев

Ведущий инженер автоматизации в «Метре квадратном»

Elasticsearch

Elasticsearch — база данных, адаптированная для полнотекстового поиска



Elasticsearch

Основные особенности:

- производительный полнотекстовый поиск
- быстрая индексация
- масштабируемая база данных
- есть возможность репликации и шардирования



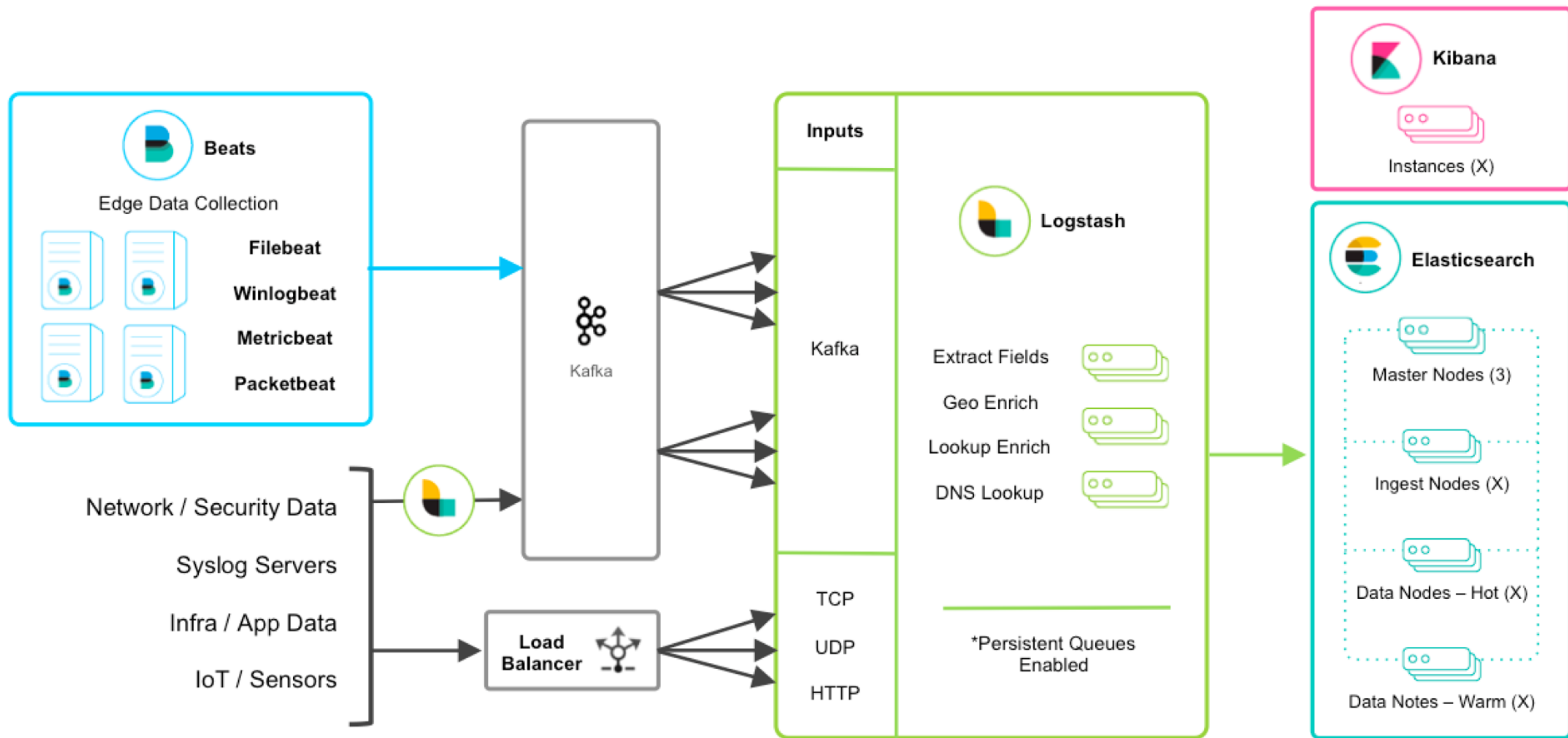
Elasticsearch

Каждый экземпляр elasticsearch может иметь ряд ролей:

- **master** — отвечает за целостность кластера и выполнение фоновых задач
- **data** — хранит данные
- **ingest** — принимает данные и трансформирует их в соответствии с заданным пайплайном
- **ml** — обрабатывает данные с использованием машинного обучения
- **coordinator** — обрабатывает входящие запросы (все экземпляры имеют эту роль)



Elasticsearch



Построение кластера

Кластерные узлы, помеченные как **hot**, хранят и собирают «быстрые» **данные логирования**, например за последние сутки.

Такие узлы требовательны к скорости операций чтения/записи диска и, соответственно, хранят данные на **SSD-носителях**.

Обычно им не требуется большой размер носителя, так как время хранения данных на таких узлах небольшое

Построение кластера

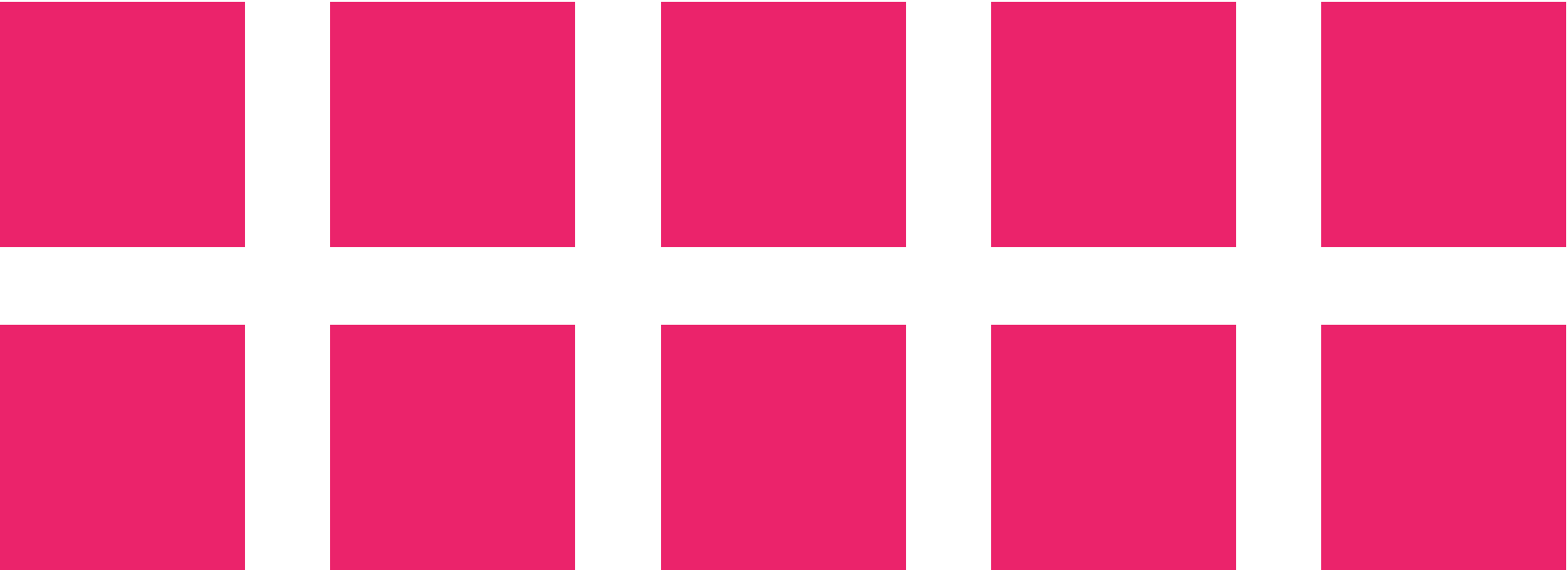
Кластерные узлы, помеченные как **warm**, хранят и собирают **«медленные» данные логирования**. Например, всё, что следует за текущими сутками.

Такие узлы не требовательны к операциям чтения/записи диска, но требовательны к объёму. Соответственно, хранить данные можно только на **HDD-носителях**.

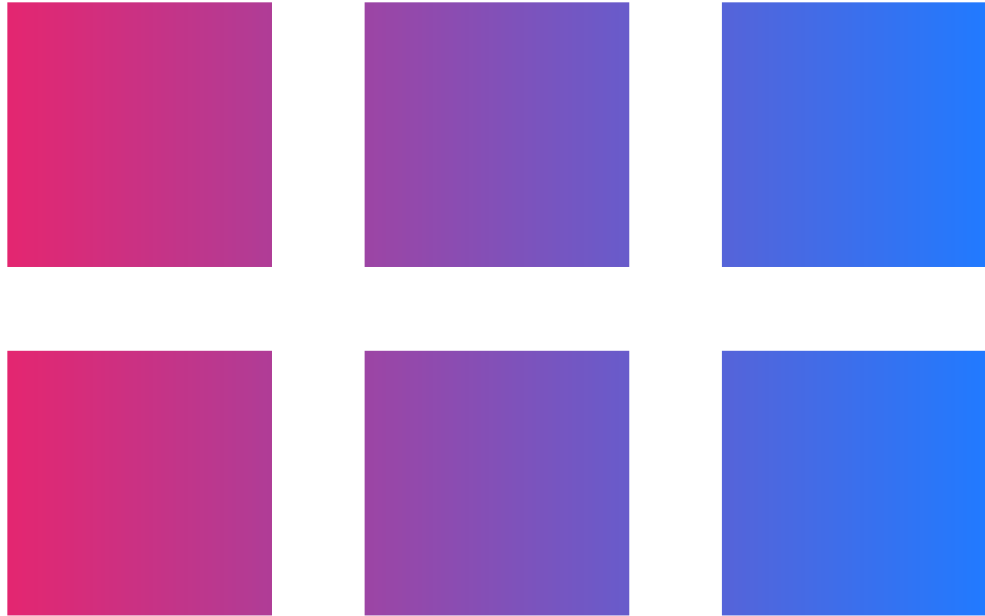
Также возможно использовать **cold**-узлы, которые хранят данные **старше 30 дней**

Построение кластера

“data”: “hot”



“data”: “warm”



“data”: “cold”



Index lifecycle management

Index lifecycle management (ILM) — процесс перемещения индексов (данных) между нодами в архитектуре Hot-Warm

Он настраивается с помощью API Elasticsearch.

Настройка процесса перемещения индекса между узлами называется политикой.

В политике можно указать триггер перемещения индекса по времени или по размеру индекса primary shard на ноде определённого типа

```
PUT /_ilm/policy/my_policy
{
  "policy":{
    "phases":{
      "hot":{
        "actions":{
          "rollover":{
            "max_size":"50gb",
            "max_age":"30d"
          }
        }
      }
    }
  }
}
```

Index lifecycle management

Чтобы **привязать индекс к политике**, необходимо указать шаблон, в котором установлено, к каким индексам применяется эта политика

В шаблоне индексов вы можете использовать **wildcard** и распространить шаблон на несколько или на все индексы.

Также в шаблоне необходимо указать псевдоним (**alias**) для индекса

```
PUT _template/my_template
{
  "index_patterns": ["test-*"],
  "settings": {
    "index.lifecycle.name": "my_policy",
    "index.lifecycle.rollover_alias": "test-alias"
  }
}
```


Index lifecycle management

Привязка псевдонимов также происходит через API Elasticsearch

Псевдонимы упрощают поиск индексов между нодами кластера и ограничивают операции записи только на индексы hot-узлов

```
PUT test-000001
{
  "aliases": {
    "test-alias": {
      "is_write_index": true
    }
  }
}
```

Пример политики hot-warm-cold

На этом этапе устанавливаем высокий приоритет индекса, чтобы горячие индексы восстанавливались раньше других при перезагрузке узла.

Через 30 дней или 50 ГБ (в зависимости от того, что наступит раньше) индекс будет обновлён (перенесён), и будет создан новый индекс.

Новый индекс запустит политику заново, а текущий индекс будет перенесён в warm-фазу

```
“hot”: {  
  “actions”: {  
    “rollover”: {  
      “max_size”: “50gb”,  
      “max_age”: “30d”  
    },  
    “set_priority”: {  
      “priority”: 50  
    }  
  }  
},
```

Пример политики hot-warm-cold

Когда индекс перейдёт в тёплую фазу:

- сократится количество его шардов до 1
- установится приоритет индекса на значение, которое ниже hot (но больше, чем cold)
- произойдёт миграция индекса на warm-узлы
- срок жизни в warm-фазе составляет 23 дня

P.S.: min_age указывает, сколько должно минимально пройти времени с создания индекса до перехода в какую-либо фазу

```
“warm”: {  
  “min_age”: “7d”,  
  “actions”: {  
    “forcemerge”: {  
      “max_num_segments”: 1  
    },  
    “shrink”: {  
      “number_of_shards”: 1  
    },  
    “allocate”: {  
      “require”: {  
        “data”: “warm”  
      }  
    },  
    “set_priority”: {  
      “priority”: 25  
    }  
  }  
},
```


Пример политики hot-warm-cold

При переходе в холодную фазу:

- Индекс заморозится. То есть станет недоступным, но будет храниться. Его можно будет использовать в будущем
- Установится минимальный приоритет восстановления индекса
- Произойдёт миграция индекса на cold-узлы
- Срок жизни индекса в cold-фазе составляет 30 дней

```
“cold”: {  
  “min_age”: “30d”,  
  “actions”: {  
    “set_priority”: {  
      “priority”: 0  
    },  
    “freeze” : {},  
    “allocate”: {  
      “require”: {  
        “data”: “cold”  
      }  
    }  
  }  
},
```

Пример политики hot-warm-cold

Эта фаза управляет **удалением индексов**

В текущей политике указано, что индекс должен быть удалён по истечении срока его жизни — 60 дней — вне зависимости от его фазы

```
“delete”: {  
  “min_age”: “60d”,  
  “actions”: {  
    “delete” : {}  
  }  
}
```

Настройка Elasticsearch

Указание типа узла происходит путём добавления конфигурации в `elasticsearch.yml` (прочие настройки стандартны для кластера Elasticsearch):

- *Hot-ноды:*

```
~]# cat /etc/elasticsearch/elasticsearch.yml | grep attr
# Add custom attributes to the node:
node.attr.box_type: hot
```

- *Warm-ноды:*

```
~]# cat /etc/elasticsearch/elasticsearch.yml | grep attr
# Add custom attributes to the node:
node.attr.box_type: warm
```

- *Cold-ноды:*

```
~]# cat /etc/elasticsearch/elasticsearch.yml | grep attr
# Add custom attributes to the node:
node.attr.box_type: cold
```


Kibana

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Kibana

Kibana – эффективное средство визуализации данных логов

В Kibana можно выводить текстовые данные и сортировать логи по определённым значениям. Например, уровням логирования.

Можно проводить агрегацию данных и визуализировать их в виде диаграмм, графиков или интерактивных карт



Kibana

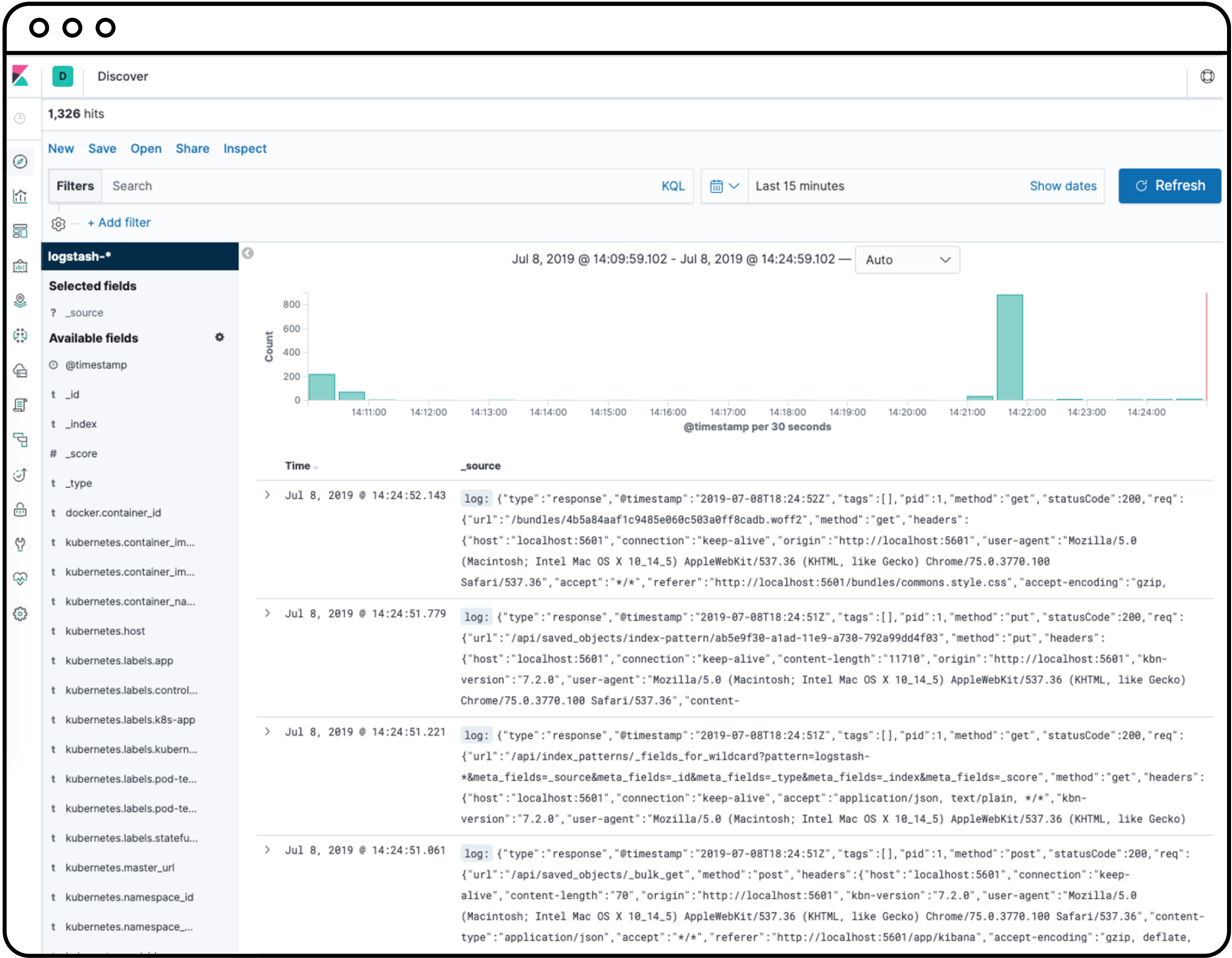
Базовая настройка Kibana довольно проста и заключается лишь в указании интерфейса, на котором она доступна, и конечных точек источников данных в виде массива сетевых параметров узлов Elasticsearch

Пример настройки Kibana:

```
server.host: "123.456.789"  
elasticsearch.url: "[https://123.456.789:9200] (http://123.456.789.9200/)"
```

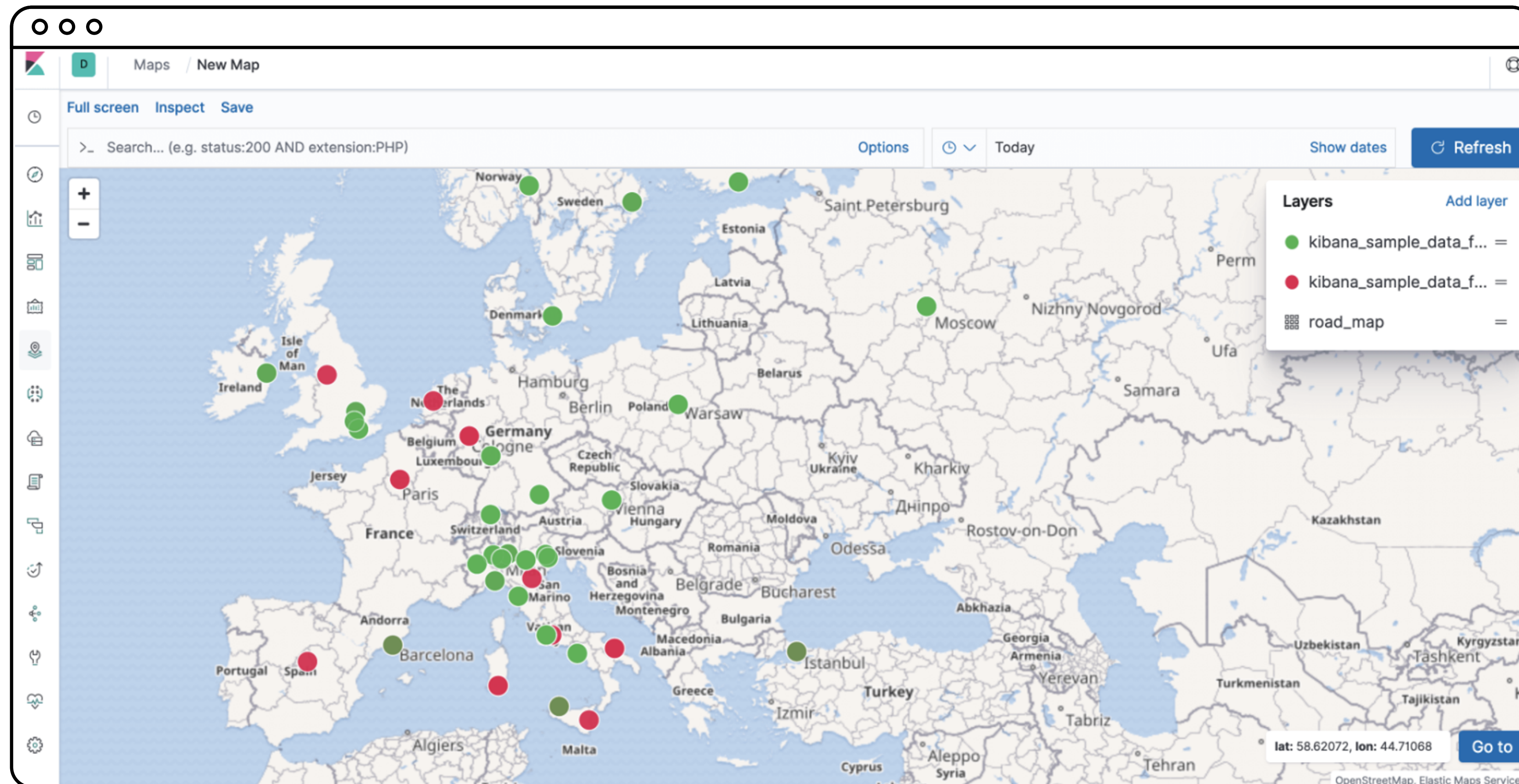
Kibana

Интерфейс поиска логов:



Kibana

Данные с геолокацией:



Kibana

Дашборды:

